

Development of Android Application for Gender, Age and Face Recognition Using OpenCV

Alen Salihbašić * and Tihomir Orehovački **

* Faculty of Economics and Tourism „Dr. Mijo Mirković“,

** Faculty of Informatics

Juraj Dobrila University of Pula, Pula, Croatia

{alsalih, tihomir.orehovacki}@unipu.hr

Abstract - The idea behind the face recognition system is the fact that every individual has a unique face. Like the fingerprint, an individual's face has many unique structures and features. Facial authentication and facial recognition are challenging tasks. For facial recognition systems to be reliable, they must work with great precision and accuracy. Images captured taking into account different facial expressions or lighting conditions allow greater precision and accuracy of the system compared to a case where only one image of each individual is stored in the database. The face recognition method handles the captured image and compares it to the images stored in the database. If a matching template is found, an individual is identified. Otherwise, the person is reported as unidentified. This paper describes and explains in detail the entire process of developing Android mobile application for recognizing person's gender, age and face. Face detection and recognition methods that have been used are described and explained as well as development tools used in the development of Android mobile application. The software solution describes the details of using the OpenCV library and shows the actual results of the mobile application through the images.

Keywords – face detection; deep neural network; face recognition; Android application; gender, age, face recognition; OpenCV; Android Studio

I. INTRODUCTION

Face recognition system is one of the most interesting and most important research fields in the last two decades. Reasons come from the need for automatic recognition and surveillance system, human interest in face recognition as well as communication between human and computer. Researches include knowledge and researchers from various research fields such as neuroscience, psychology, computer vision, pattern recognition, image processing and machine learning.

The first step to a reliable face recognition is detection of locations of faces in images where faces are present. Face detection is considered as a fundament to face recognition. However, it is a challenging task for computer system on which can affect issues such as location of the face, person orientation, face expressions, partial face coverage, lighting conditions etc.

This paper will explore the possibility of implementing a face recognition system, as well as gender and age recognition, onto a mobile device in the form of an Android application with the help of OpenCV library.

The remainder of the paper is structured as follows. Biometric system is briefly described in second section. Used algorithm for face detection and face recognition is explained in third section. The design and application functionalities are described in fourth section. Software solution is presented and thoroughly explained with code snippets and results in fifth section. Conclusions are drawn in last section.

II. BIOMETRIC SYSTEM

The biometric system is essentially a pattern recognition system that makes personal identification by determining the authenticity of a certain physical or behavioral characteristic possessed by the user. The biometric system can be divided into the data enrollment module and the identification module.

During the enrollment phase, the biometric characteristic of an individual is scanned by a biometric sensor in order to gain a digital representation of the characteristic. In order to facilitate matching and to reduce storage requirements, the digital representation is further processed by feature extractor to create a compact but expressive representation, called a template. During the recognition phase, the biometric sensor captures the characteristics of an individual to be identified and converts it into a digital format, which is further processed by a feature extractor to get the same representation as the template. The result of representation is entered in a feature matcher which compares it against the templates to determine the identity of an individual [4].

III. FACE DETECTION AND RECOGNITION

With the regard to the image, the purpose of face detection is to determine whether the face is in the image and, if it is present, to restore the location of the face in the image and the scope of each face. There are many related face detection issues. Some of the difficulties can be determining the position of the face in the image, detecting the presence and location of facial features such as the eyes, nose, eyebrows, nostrils, mouth, lips and ears, as well as face authentication that confirms the individual's identity in the image, assessing its location and orientation in real time and recognizing the emotional state of the face. Accordingly, the conclusion is that face detection is the first step in any automated system that addresses the above-mentioned problems [8].

Local Binary Patterns (LBP) is a very efficient texture operator that is defined as a grayscale invariant texture measure, derived from a general definition of texture in a local neighborhood. Due to its discriminatory power and computational simplicity, LBP texture operator has become popular approach in various applications. It can be seen as a unique approach to traditional statistical and structural patterns of texture analysis. Perhaps the most important feature of an LBP operator in real world applications is its resistance to monotonic grayscale changes caused, for example, by variations in illumination. Another important feature is its computational simplicity, which enables the analysis of images in challenging real-time circumstances [7].

The original LBP operator forms labels for the image pixels by thresholding the 3x3 neighborhood of each pixel with the centre value and considering the result as a binary number. The histogram of these different labels can then be used as a texture descriptor [3]. Before applying LBP operation, it is necessary to train the algorithm with dataset of the facial images of the person we want to recognize, as well as, set an ID for each image in order for algorithm to use that information in output result.

As shown in Figure 1, on the facial grayscale image example, LBP operator takes 3x3 pixels part of the image which can be also represented as 3x3 matrix containing the intensity of each pixel (0-255). The central value of the matrix is used as threshold and to define the new values from the 8 neighbors. For each neighbor of the threshold, a new binary value is set. 0 if the value is lower than the threshold or 1 if the value is equal or higher than the threshold. The matrix will contain only binary values which if concatenated form a new binary value. Then, the new binary value is converted to a decimal value and set it to the central value of the matrix,

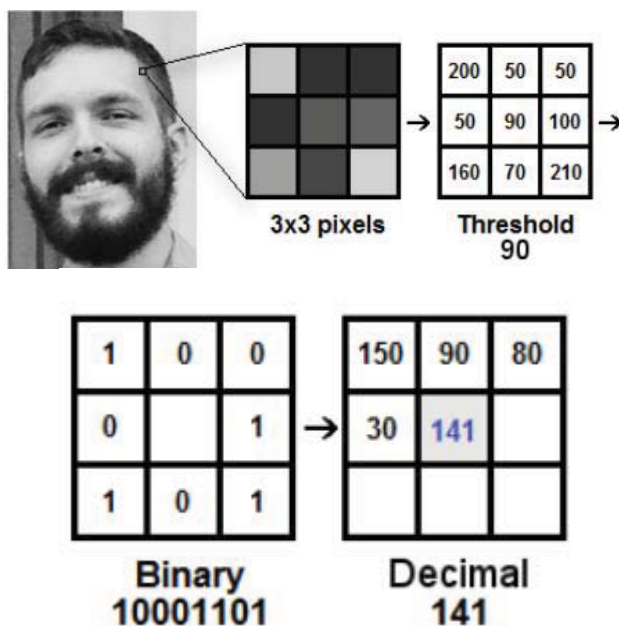


Figure 1. Example of LBP operation [8]

which is actually a pixel from the original image. At the end of operation, a new image is created which represents better the characteristics of the original image.

LBPH (Local Binary Pattern Histogram) algorithm uses four parameters; radius, neighbors, grid X and grid Y. The radius is used to build the circular local binary pattern and represents the radius around the central pixel, it is usually set to 1. Neighbors represents the number of sample points used to build the circular local binary pattern, it is usually set to 8. Grid X represents the number of cells in the horizontal direction, it is usually set to 8. Grid Y represents the number of cells in the vertical direction, it is usually set to 8.

Extracting the histograms for face recognition, the image is divided into multiple grids with grid X and grid Y parameters, as shown in Figure 2. Each histogram from each grid will contain 256 positions representing the occurrences of each pixel intensity. Final histogram is created with concatenation of each histogram and represents the characteristics of the original image. When performing face recognition, each final histogram is used to represent each image from the training dataset against final histogram of given input image. Comparing two histograms, input image and training image, algorithm returns the image with the closest histogram[8].

IV. DESIGN AND FUNCTIONALITIES

Use case diagram, shown in Figure 3, describes the design of interaction with mobile application where all possible options of interacting with mobile application are displayed and described.

The home screen shows the option of choosing three different actions, gender, age, or face recognition. Pressing any desired action triggers the selected activity. At the start of each activity, a camera is initialized in order to process real-time images. Also, by initializing

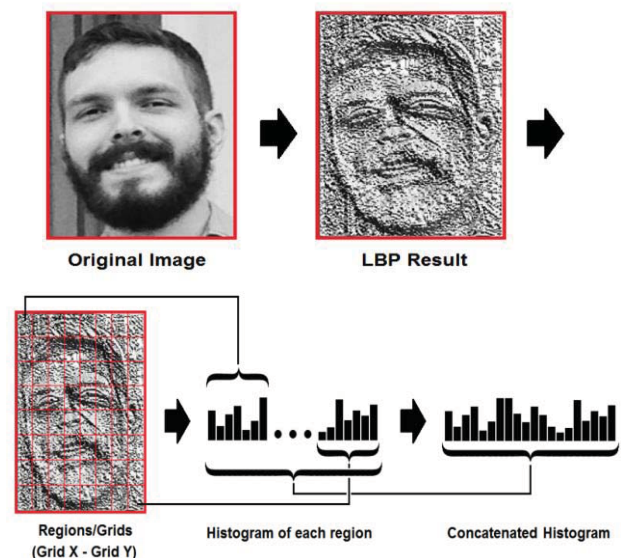


Figure 2. Example of extracting histograms [8]

the camera, face detection classifier loads in the background of mobile application. When gender or age recognition activity starts, the data of pretrained model for gender and age recognition is loaded in the background.

Application supports simultaneous detection of all faces, but does not support simultaneous recognition of all faces due to limited mobile device resources. By selecting face recognition, a training activity is initiated in which the user has to take a certain number of facial images of the person (s)he wants to recognize. In training activity, it is possible to capture and save images of faces in device memory, to create a face recognition model and automatically save extracted facial features in device memory, to delete all images of faces, as well as to delete facial features classifier and to run face recognition activity after fulfilling certain above mentioned conditions. In the recognition process, only one face must be in the camera frame, and then the process of recognizing gender, age or face is automatically initialized.

The face detection result is visible as rectangular shape scoped around each detected face in the camera frame and the recognition results are written, accordingly to their activity, above the rectangle that appears around the detected face. Possible values for gender recognition outcome are; „MALE, FEMALE“. When recognizing the age, the possible values of the outcome may be; „0-2, 4-6, 8-13, 15-20, 25-32, 38-43, 48-53, 60+“. In face recognition, the value of the outcome is actually the image number that has the highest probability of matching the person's face in camera frame.

V. SOFTWARE SOLUTION

Every interaction with the camera is performed through OpenCV abstract class, CameraBridgeViewBase.

The task of class is to interact with the camera and OpenCV library. The main responsibility is controlling when the camera can be enabled, camera frame processing, calling external interfaces for any camera frame adjustments and rendering camera frame results to the mobile device display. Interface that enables starting, stopping and camera frame manipulating is CvCameraViewListener2 and its methods, onCameraViewStarted(), onCameraViewStopped() and onCameraFrame(). Method onCameraViewStarted() is invoked when camera preview has started and then the frames are delivered to client via onCameraFrame() callback. Method onCameraViewStopped() is invoked when camera preview has been stopped for some reason and after no frames will be delivered to client through onCameraFrame() callback. Method onCameraFrame() is invoked when delivery of the frame needs to be done and the returned value is a modified frame which needs to be displayed on the screen. It takes CameraBridgeViewBase.CvCameraViewFrame input frame which represents a single frame from camera for onCameraFrame() callback and it must be manipulated by its default methods, gray() or rgba(), and returns specified Mat object with frame.

In order to be able to access mobile device camera, permission rights in file AndroidManifest.xml have been added. For face detection, OpenCV object detection class CascadeClassifier has been used, which allows us to load previously evaluated cascade classifiers. Face detection was achieved with the help of trained LBP face features classifier in the form of XML file.

A. Face detection

In order for face detection to be successful, the current camera frame uses Mat class in real time, which is

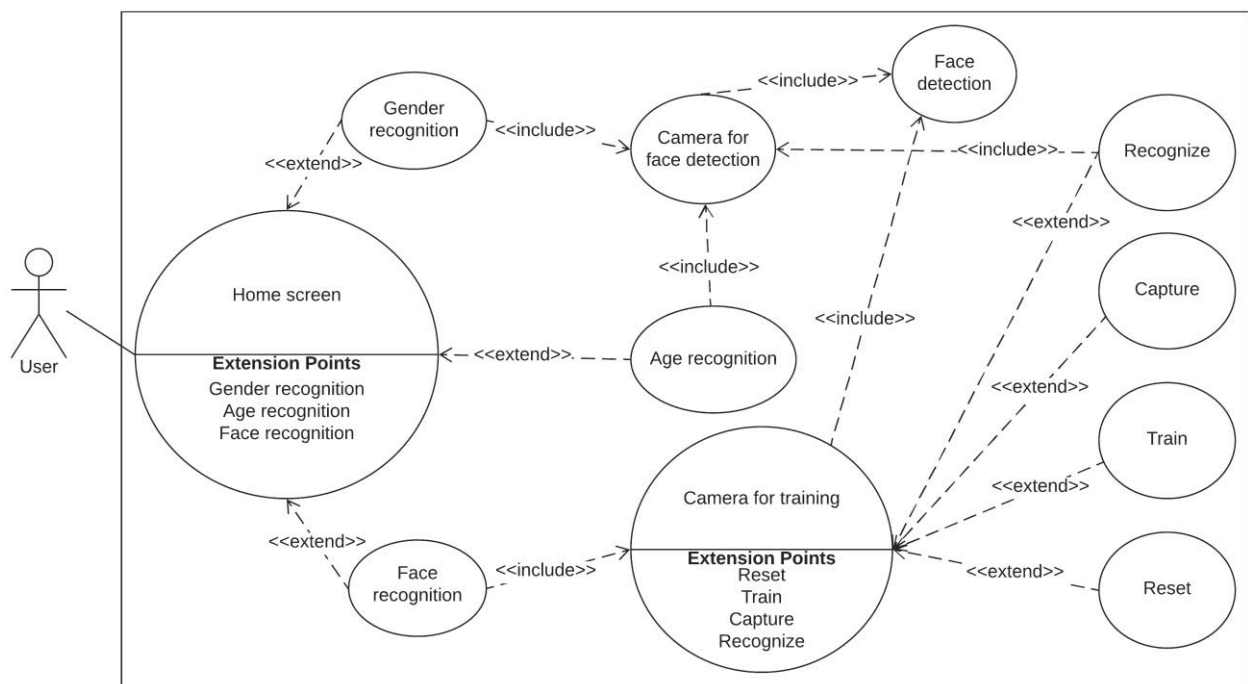


Figure 3. Use case diagram of an Android application for gender, age and face recognition

used in all OpenCV 2D computer vision applications. Mat class is actually basic image container. It can appear as a class with two parts of data. Matrix header, which contains information such as matrix size, method of storage usage, address of stored matrix and matrix pointer, which contains pixel values.

Using detectMultiScale() method, see Figure 4, we can detect faces of various sizes in a given camera frame, and save them to MatOfRect object. In order to be able to draw a rectangle around each of the detected faces in the image, all detected faces are stored in a 2D array. Drawing a rectangle around the detected face is performed by using the method rectangle() of OpenCV image processing module, Imgproc.

B. Gender and age recognition

Gender and age recognition are using Levi and Hassner's Caffe model [5]. The model is trained on the Adience collection of unfiltered faces for gender and age classification and contains 26.580 images of a total of 2.284 subjects. The source for the photos in Adience collection are the Flickr.com albums, produced by automatic upload from smartphones iPhone 5 or later that are publicly available through the Creative Commons (CC) license. All images were manually labeled for age and gender using both the images themselves and using any available contextual information (image tags, associated text, additional photos in the same album, etc.) [2].

The convolutional neural network contains three convolutional layers, each followed by a rectified linear operation and a pooling layer. The first convolutional layer contains 96 filters of 7x7 pixels, the second convolutional layer contains 256 filters of 5x5 pixels, the third and final convolutional layer contains 384 filters of 3x3 pixels. Finally, two fully-connected layers are added, each containing 512 neurons. At the end, the result is obtained from fully-connected layers which is the class attribute, in this case gender or age, to which the input image belongs [5]. The model was implemented with OpenCV module for deep neural networks, Dnn. The gender recognition result is described as the output value of 0 or 1, where 0 indicates a male person, and the value 1 indicates a female person. In age recognition, the result of the age estimation is described in the form of output values from 0 to 7, where each value represents a particular age group; 0-2, 4-6, 8-13, 15-20, 25-32, 38-43,

```
MatOfRect faces = new MatOfRect();

//Using detection classifier
if (mFaceDetector != null) {
    mFaceDetector.detectMultiScale(mGray,
        faces, 1.1, 5, 2, new Size(mAbsoluteFaceSize,
        mAbsoluteFaceSize), new Size());
} else {
    Log.e(TAG, "Detection is not selected!");
}

//Drawing rectangle around detected face
Rect[] facesArray = faces.toArray();
for (int i = 0; i < facesArray.length; i++) {
    Imgproc.rectangle(mRgba, facesArray[i].tl(),
        facesArray[i].br(), new Scalar(0, 255, 0, 255), 3);
}
```

Figure 4. Face detection and drawing rectangles around detected

48-53, 60+.

As shown in Figure 5, method for age recognition as parameters takes an RGBA frame from a mobile camera and a 2D array of detected faces. Within the method, a new Mat object is created that contains only the detected face within the entire frame. Then, the facial image is decreased to the required image resolution of the convolutional neural network we use, 227x227, and switched to the three-channel image of the BGR format. After the preliminary image processing, the image is sent through a deep neural network where the result of the highest value is retrieved via the static class MinMaxLocResult, which prints the gender or age group class number that the detected face belongs to and returns the result as a String variable. The result of age recognition is presented in Figure 6.

The only difference between age and gender recognition methods is the output result, as gender recognition has output values 0 or 1. The result can be seen in Figure 7.

C. Face recognition

Before it is possible to recognize a face, it is necessary to train a face recognition model with face images of a person which is going to be recognized. By pressing Capture on the mobile device display, the method for taking detected facial images is performed. The method parameters, as shown in Figure 8, are the image number, the RGBA frame and the face detection classifier. At the very beginning of the method execution, a directory called FacePics is created in the device memory where the face images will be stored, as well as the trained face recognition model in XML format. It follows with the preliminary image processing, converting RGBA frame to a grayscale image, detecting and extracting only a face from the overall image, reducing the resolution of the face image to 92x112 and equalizing face image histogram. Preliminary image

```
//Method for age recognition
private String predictAge(Mat mRgba,
    Rect[] facesArray) {
    try {
        for (Rect face : facesArray) {
            Mat capturedFace = new Mat(mRgba, face);
            //Resizing pictures to resolution of Caffe model
            Imgproc.resize(capturedFace, capturedFace,
                new Size(227, 227));
            //Converting RGBA to BGR
            Imgproc.cvtColor(capturedFace, capturedFace,
                Imgproc.COLOR_RGBA2BGR);
            //Forwarding picture through Dnn
            Mat inputBlob = Dnn.blobFromImage(capturedFace,
                1.0f, new Size(227, 227), new Scalar(78.4263377603,
                87.7689143744, 114.895847746), false, false);
            mAgeNet.setInput(inputBlob, "data");
            Mat probs = mAgeNet.forward("prob").reshape(1, 1);
            //Getting largest softmax output
            Core.MinMaxLocResult mm = Core.minMaxLoc(probs);
            //Result of age recognition prediction
            double result = mm.maxLoc.x;
            Log.i(TAG, "Result is: " + result);
            return AGES[(int) result];
        }
    } catch (Exception e) {
        Log.e(TAG, "Error processing age", e);
    }
    return null;
}
```

Figure 5. Age recognition method

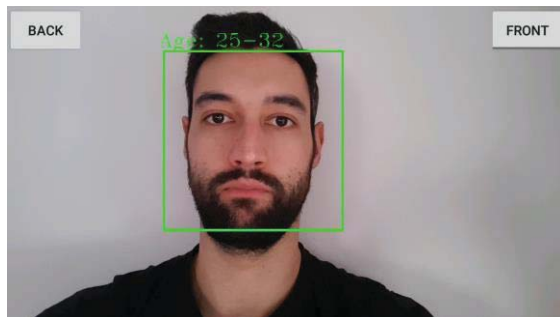


Figure 6. The result of age recognition

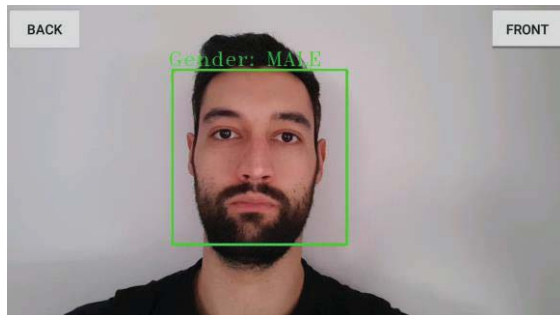


Figure 7. The result of gender recognition

processing is performed to optimize resources and performance required for computational training of face recognition algorithm. The face image is saved into the directory FacePics of the Pictures directory using the OpenCV read and write image module, Imgcodecs.

After capturing facial images, next actions are image processing for training face recognition model, creating a face recognition model, face recognition model training with facial images and corresponding facial labels and saving trained face recognition model to device memory. Each saved image in the FacePics directory is loaded into

a grayscale and is saved in the Mat object. The label of each image is actually the number of the image. The processing of the input image is followed, and finally, each image is saved in the MatVector object with the corresponding label being written to the Mat object.

The mobile application uses the LBPH face recognition model that is created using a static FaceRecognizer class located in the OpenCV face analysis module, Face. After creating a face recognition model, it is possible to use its methods. One of the methods is a train() method that as a parameter requires a set of images to be used to train the model and the corresponding labels of these images. After training a face recognition model, the model is saved in the device memory of the mobile device by the write() method. The result is a file in XML format that contains all of the extracted features of the face images over which the training was performed. The file is saved in the directory FacePics.

After face recognition model training has been completed, by pressing the Recognize button on the mobile device display, the face recognition activity will launch. When the face recognition activity is started, in the background is loaded trained face recognition model in XML format. Trained face recognition model is used to recognize detected faces on a mobile device display. Recognition shown in Figure 9 is performed automatically on the mobile device display if only one face is in a camera frame. Prior to performing the recognition method, preliminary image processing is required. Face recognition is performed by the method predict() of static class FaceRecognizer. In the method predict(), as the parameters are, the image over which the face recognition is executed, list of image labels, and the probability of matching the detected face with the face

```
//Method for capturing photos
public static void takePhoto(int photoNumber,
    Mat rgbMat,
    CascadeClassifier faceDetector)
throws Exception {
    File facePicsPath = new File(String.valueOf(ROOT));
    if (facePicsPath.exists() && !facePicsPath.isDirectory())
        facePicsPath.delete();
    if (!facePicsPath.exists())
        facePicsPath.mkdirs();

    Mat grayMat = new Mat();
    //Converting RGBA to GRAY
    Imgproc.cvtColor(rgbMat, grayMat, Imgproc.COLOR_RGBA2GRAY);
    MatOfRect detectedFaces = new MatOfRect();
    faceDetector.detectMultiScale(grayMat, detectedFaces);
    Rect[] detectedFacesArray = detectedFaces.toArray();
    for (Rect face : detectedFacesArray) {
        Mat capturedFace = new Mat(grayMat, face);
        //Resizing to 92x112
        Imgproc.resize(capturedFace, capturedFace,
            new Size(IMG_WIDTH, IMG_HEIGHT));
        //Histogram equalizing
        Imgproc.equalizeHist(capturedFace, capturedFace);
        if (photoNumber <= PHOTOS_TRAIN_QTY) {
            File savePhoto = new File(facePicsPath,
                String.format("%d.png",
                    photoNumber));

            savePhoto.createNewFile();
            //Saving photos to directory FacePics
            Imgcodecs.imwrite(savePhoto.getAbsolutePath(),
                capturedFace);
            Log.i(TAG, "PIC PATH: " + savePhoto.getAbsolutePath());
        }
    }
}
```

Figure 8. Method for capturing photos

```
//If one face is detected, face prediction is executed
if (facesArray.length == 1) {
    try {
        //Conversion from OpenCV Mat to JavaCV Mat
        opencv_core.Mat javaCvMat = new opencv_core.Mat((Pointer) null)
        {{address = mGray.getNativeObjAddr();}};
        //Picture resizing
        resize(javaCvMat, javaCvMat,
            new opencv_core.Size(Methods.IMG_WIDTH,
                Methods.IMG_HEIGHT));
        //Histogram equalizing
        equalizeHist(javaCvMat, javaCvMat);
        IntPointer label = new IntPointer(1);
        DoublePointer confidence = new DoublePointer(1);
        mLBPFaceRecognizer.predict(javaCvMat, label, confidence);
        int predictedLabel = label.get(0);
        double acceptanceLevel = confidence.get(0);
        String name;
        Log.d(TAG, "Prediction completed, predictedLabel: "
            + predictedLabel + ", acceptanceLevel: "
            + acceptanceLevel);
        if (predictedLabel == -1 || acceptanceLevel >= THRESHOLD) {
            name = "-";
        } else {
            name = Integer.toString(predictedLabel);
        }
        //The result of face recognition
        for (Rect face : facesArray) {
            int posX = (int) Math.max(face.tl().x - 10, 0);
            int posY = (int) Math.max(face.tl().y - 10, 0);
            Imgproc.putText(mRgba, "Closest picture: num." +
                name, new Point(posX, posY),
                Core.FONT_HERSHEY_TRIPLEX, 1.5,
                new Scalar(0, 255, 0, 255));
        }
    } catch (Exception e) {
        Log.d(TAG, e.getLocalizedMessage(), e);
    }
}
```

Figure 9. Performing face recognition

from the FacePics directory. The face recognition model uses the closest neighbor recognition method.

In order to identify the number of the closest recognized image, the detected face must meet certain conditions of trained face recognition model. There are two conditions; the first condition is that the detected face on the mobile device display has face features which are compared to face features found in the trained face recognition model, and the second condition is that the probability of the detected face does not exceed the programmed threshold value (130.0). A face recognition results may be negative, if one or both of the conditions are not met, or positive, showing the number of the closest recognized face image from the FacePics directory. The negative and positive results are shown respectively in Figure 10 and 11.



Figure 10. Negative output of face recognition



Figure 11. Positive output of face recognition

VI. CONCLUSION

This paper in detail explains the development of a gender, age and face recognition system on an Android mobile device. The overall software solution for face detection and recognition in the mobile application is achieved with the help of the OpenCV library which provides a numerous functionalities for developing computer vision applications. A LBP face features classifier was used for face detection and LBPH model was used for face recognition purposes. Gender and age recognition was achieved by populating a deep neural network that allows the use of recognition functions without capturing the images and training gender and age recognition models. Although recognition algorithms successfully perform job of face recognition, they are affected by various conditions such as illumination, pose of the person, facial expressions, face coverage, camera features as well as the performance of the mobile device itself. It is shown that in spite of all the above mentioned issues, it is possible to implement the recognition system on the Android mobile device.

REFERENCES

- [1] E. Eiding, R. Enbar, T. Hassner, "Age and Gender Estimation of Unfiltered Faces", Transactions on Information Forensics and Security (IEEE-TIFS), 12th ed., vol. 9., pp.2170-2179., 2014.
- [2] A. Jain, L. Hong and S. Pankanti, "Biometric Identification", Communications of the ACM, 2nd ed., vol. 43., pp.91-98., 2000.
- [3] K. Kushsairy, K. Kamaruddin, M. & N. Haidawati, I S. Sairul, B. Zulkifli, "A comparative study between LBP and Haar-like features for Face Detection using OpenCV", Conference: 2014 4th International Conference on Engineering Technology and Technopreneuship (ICE2T), pp.335-339., 2014.
- [4] G. Levi and T. Hassner, "Age and Gender Classification Using Convolutional Neural Networks", Boston: IEEE Workshop on Analysis and Modeling of Faces and Gestures (AMFG), 2015.
- [5] M. Pietikäinen, "Local Binary Patterns", Scholarpedia, 3rd ed., vol. 5., pp.9775., 2010.
- [6] Q. M. Rizvi, B. G. Agarwal and R. Beg, "A Review on Face Detection Methods", Journal of Management Development and Information Technology, vol. 11., 2011.
- [7] A. Salihbašić, "Razvoj Android aplikacije za prepoznavanje lica" (in Croatian), Master's thesis, Juraj Dobrila University of Pula, Faculty of Economics and Tourism „Dr. Mijo Mirković“, 2018.
- [8] K. Salton do Prado, „Face Recognition: Understanding LBPH Algorithm“, www.towardsdatascience.com, 2017.